INTERVIEW SUBMISSION

GPMS Frontend Engineering Deep Dive

Implementation-level process and architecture report based on the current production frontend codebase.

Project	GPMS (Garment Production Management System) - Web Frontend
Frontend Stack	Next.js 15 App Router, React 19, TypeScript, React Query, MUI, Tailwind, Zustand
Live Frontend	https://gpms-web664.vercel.app
API Base	https://clever-serenity-production.up.railway.app/api
Swagger	https://clever-serenity-production.up.railway.app/api/docs
Reference Date	March 5, 2026
Prepared For	Frontend engineering interview evaluation

> This report focuses on depth: architecture rationale, domain workflow mapping, technical tradeoffs, quality posture, and explicit next-step hardening opportunities.

1. What I Built and Why

GPMS Web is an operational system, not a brochure UI. Users execute daily manufacturing workflows: create master data, create and progress POs, plan production lines, capture output, run QC, and monitor plan-vs-actual in near real time.

I built the frontend around one core requirement: **workflow correctness under fast-moving factory usage**. The architecture is intentionally shaped to keep context consistent (factory/date), prevent accidental drift, and align user actions with backend business rules.

Engineering goals

- Keep operational context globally consistent across routes.
- Ensure every primary user action maps cleanly to a backend transition.
- Provide high-velocity interaction patterns (quick forms, drawers, command palette, CTA routing).
- Retain implementation clarity for future contributors.

Business process covered end-to-end

```
Buyer -> Style/BOM -> PO -> Material Requirement -> Inventory -> Plan ->
Cutting -> Sewing -> QC -> Shipment/Closure
```

Stage	Primary Routes	Main Outcome
Master Data	`/buyers`, `/materials`, `/styles`	Establish clean, reusable operating data
Order Management	`/pos`	Create/manage PO lifecycle and requirement visibility
Supply Control	`/inventory`	Receive/issue stock with current position visibility
Execution Planning	`/plans`	Assign PO items to lines with dated targets
Floor Execution	`/cutting`, `/sewing`, `/qc`	Track bundle progress, hourly output, and quality results

Stage	Primary Routes	Main Outcome
Control Tower	`/dashboard`	Aggregate KPIs and fast cross-module actions

2. Frontend System Architecture

2.1 App composition

- **Root layout:** fonts, MUI App Router cache provider, global providers.
- **Protected layout:** every operational route wrapped by auth gate.
- **Shell pattern:** persistent sidebar + topbar + command palette + contextual right drawer.

2.2 Provider stack

- `QueryClientProvider` for server state/caching/invalidation.
- `ThemeProvider` for dark-first MUI theming.
- `AuthProvider` for session hydration and factory bootstrap.
- `Toaster` for non-blocking operation feedback.

2.3 Auth and access model

- Client hydrates session with `GET /auth/me` on startup.
- Unauthenticated access redirects to `/login?next=...`.
- Factory is selected from persisted factory, `activeFactory`, or first accessible factory.
- Logout calls backend endpoint then clears local auth/factory context.

2.4 Context and state strategy (Zustand)

State is split by durability requirements:

- Persisted: `factoryId`, theme, sidebar collapsed state.
- Non-persisted by design: `selectedDate` (resets to local today at page load).
- Ephemeral UX state: command palette open, action intent, global search text.

> **Key decision:** date is intentionally non-persistent to avoid stale-day execution errors when users return the next day.

2.5 API client contract

- Axios with `withCredentials: true` for cookie auth.
- Interceptor injects `x-factory-id` from store into every scoped request.
- Defensive request normalization for `/plans/:id/assign` payload shape.
- Thin wrappers `apiGet` and `apiPost` keep page code concise.

3. UX System and Interaction Process

3.1 Navigation and orchestration

- Sidebar exposes all manufacturing domains in one stable frame.
- Topbar standardizes cross-page context controls: factory, date, global search.
- Command palette (`Cmd/Ctrl + K`) provides fast route switching and action intents.
- Action-intent drawer maps quick commands to the right module (PO, buyer, sewing, QC).

3.2 Page-level interaction pattern

Most modules follow a repeatable pattern:

- Left panel: create/update forms.
- Main panel: table/grid of current records.
- Detail drawer: focused edits, progression actions, or drill-down analytics.
- React Query invalidation after mutation to keep view coherent.

3.3 Real-time visibility design

- Dashboard polls every 10 seconds for plan/line/QC snapshots.
- Sewing status also polls every 10 seconds for floor visibility.
- Dashboard includes optimistic output posting for immediate operator feedback.
- Live timestamp and seconds-since-update indicator communicate freshness explicitly.

3.4 Responsive behavior

- Desktop shell with collapsible left navigation.
- Mobile overlay nav using animated sheet and backdrop dismiss.
- Table + drawer approach preserves detail access without dense routing complexity.

4. Domain Module Walkthrough (Implementation Depth)

4.1 Buyers

- Create, update, delete buyer records.
- Global search integrates with local filtered table.
- Detail drawer supports in-place edits and destructive action.

4.2 Styles and BOM

- Style CRUD with dedicated BOM editor in same flow.
- BOM supports multi-line material entries.
- Client prevents duplicate material IDs in a BOM before submit.
- Save operation sends full `items[]` payload, matching backend replace semantics.

4.3 Materials

- Material creation (name/type/UOM).
- Serves as upstream data for BOM and inventory calculations.
- Drawer presents quick profile details for selected material.

4.4 Purchase Orders

- Create PO header, add line items, confirm draft, update lifecycle status.
- Status controls appear in both registry table and detail drawer.
- Material requirement panel compares required qty vs current stock.
- CSV export for operations handoff (`Material, Required, Stock, UOM, Shortage`).

4.5 Inventory

- Receive operation posts `refType: "GRN"`.
- Issue operation posts `refType: "ISSUE_CUTTING"`.
- Shared material selector reduces input errors and aligns IDs with stock records.

4.6 Planning

- Create named plan with start/end dates.
- Assign PO item to line with assignment range and dated target payload.
- Plan detail table and drawer expose assignment and targets for review.
- Client pre-validates date order before API submission.

5. Production Floor Modules

5.1 Cutting

- Select PO then PO item to create cutting batch.
- Generate bundles by size + quantity against selected batch.
- Batch and bundle details are queryable and shown in drawer drill-down.

5.2 Sewing

- Post hourly output by line/date/hour/qty with optional bundle linkage.
- Date field is tied to global date to prevent module drift.
- Status table shows total output, per-hour split, and WIP composition.
- Line drawer expands hourly + WIP breakdown for focused review.

5.3 QC

- Submit inspection with type, pass/fail, optional notes, optional defect rows.
- QC summary query by selected date returns pass rate, DHU estimate, and top defects.
- Bundle selector allows existing bundle choices and custom typed IDs.

5.4 Dashboard

- Aggregates three data streams concurrently: `/reports/plan-vs-actual`, `/sewing/line-status`, `/qc/summary`.
- Computes and displays derived KPIs (planned, produced, efficiency, WIP, defect rate).
- Provides quick output form and CTA routing into PO, Sewing, and QC flows.
- Line monitor cards open a detail drawer with hourly chart + WIP chips + top defects.

5.5 Example mutation flow (dashboard quick output)

```
1) Validate line/hour/qty locally
2) Cancel in-flight dashboard query
3) Snapshot previous dashboard cache
4) Apply optimistic increment to line + plan rows
5) POST /sewing/hourly-output
6) On error: rollback cache snapshot + toast
7) On settle: invalidate dashboard query for server truth
```

6. Engineering Quality: Validation, Resilience, and Tradeoffs

6.1 Validation strategy

Common validation is centralized in `lib/form-validation.ts`:

- `requireText` for mandatory string fields.
- `requireDate` and `ensureDateOrder` for date-safe workflows.
- `requirePositiveNumber` with configurable minimum (including hour slot lower bound of 0).

6.2 Error handling contract

- Mutations use consistent `onError` parsing of backend message first, fallback to generic.
- Toasts communicate action status without blocking the operator.
- Detail/list queries are invalidated after successful mutations to preserve consistency.

6.3 Where correctness is intentionally backend-owned

- PO state transition legality (including shipping/closing readiness).
- Factory authorization and access validation.
- Inventory sufficiency checks for issue operations.
- QC and production constraints.

> **Design position:** frontend accelerates the workflow, backend owns final authority. This avoids client-side state assumptions becoming business truth.

6.4 Performance posture

- React Query caching prevents unnecessary refetch thrash.
- Parallel API loading on dashboard with `Promise.all`.
- Optimistic updates for immediate response in high-frequency output entry.
- UI animations are scoped and lightweight (framer-motion for transitions, not heavy layout loops).

6.5 Known technical debt

- Residual `any` in error handlers and a few UI wrappers.
- One hook dependency warning in `resource-page.tsx`.

- Theme toggle currently sets document color scheme to dark in provider effect.

7. Build and Lint Evidence (Executed March 5, 2026)

7.1 Lint

Command: `pnpm lint`

0 errors 26 warnings

Most warnings are `@typescript-eslint/no-explicit-any` and a small set of unused vars / hook dependency notices.

7.2 Production build

Command: `pnpm build`

Build succeeded Next workspace-root warning only

7.3 Route footprint from build output

Route	Page Size	First Load JS
`/dashboard`	110 kB	403 kB
`/pos`	6.24 kB	302 kB
`/plans`	3.35 kB	299 kB
`/styles`	3.34 kB	299 kB
`/cutting`	2.77 kB	298 kB
`/inventory`	2.52 kB	298 kB
`/qc`	2.59 kB	298 kB
`/sewing`	2.64 kB	298 kB

> The dashboard is expectedly the heaviest route due to charting, multi-query aggregation, and monitoring widgets. This is acceptable for current scope but should be watched as module complexity grows.

8. Example End-to-End Scenario (Operator Journey)

Scenario: from new order to shipment readiness

1. Create buyer, materials, style, and multi-line BOM.
2. Create PO and add items.
3. Confirm PO.
4. Review PO material requirement against current stock table.
5. Receive missing stock in inventory.
6. Create production plan and assign PO item to line with target date/qty.
7. Create cutting batch and generate bundles.
8. Post sewing hourly outputs against line and bundle.
9. Submit QC inspections and defect records.
10. From PO module, execute Ship then Close transitions once backend conditions are satisfied.

Frontend mechanisms that make this reliable

- Shared date/factory context means every stage refers to one operating slice.
- Drawer details reduce context switching while retaining full record depth.
- Dashboard gives real-time aggregate confidence before progression actions.
- Error toasts expose backend rule failures in plain workflow context.

9. What This Demonstrates About My Engineering Depth

- **System thinking:** I mapped UI by manufacturing value stream, not by isolated screens.
- **Architecture discipline:** I enforced global context and clean request contracts.
- **Execution quality:** I implemented high-velocity UX patterns while preserving backend truth.
- **Operational realism:** I handled polling, optimistic updates, and lifecycle transitions where day-to-day users feel friction.
- **Technical honesty:** I can clearly identify debt and hardening priorities, not only feature output.

10. Next Hardening Plan

- Replace all residual `any` with typed API error envelopes.

- Add integration tests for critical chain: PO -> Plan -> Cutting -> Sewing -> QC -> Ship/Close.
- Add role-aware UI constraints to mirror backend role model for operator clarity.
- Split dashboard heavy widgets via dynamic imports if bundle pressure grows.
- Promote lint warnings to errors after cleanup sprint.

Final note: the current frontend is production-capable for the defined workflow scope and structured for maintainable growth.